

```

In [1]: # Capstone Project: Intelligent E-Commerce Pricing Engine
# Student Name : Preetham Sharvin Danthi
# Domain : Data Science / Aug 1st Batch

# Problem Statement

# Online retailers often struggle to set competitive prices because competitor pricing changes frequently.
# Manual tracking of competitor prices is time-consuming and inefficient.
# This project aims to build an automated pricing pipeline to support better pricing decisions.

#Main Objectives:

# Scrape competitor book data to understand current market trends.
# Clean and preprocess raw scraped data into a structured dataset.
# Build a Machine Learning model to predict estimated book price using rating and stock availability.
# The final goal is to provide a simple data-driven system for intelligent e-commerce pricing.

In [ ]: # Phase 1: Data Acquisition (Web Scraping)

# Objective: My goal in this phase was to automatically collect details of nearly 1,000 books from books.toscra

# Code Logic:

# I used requests to access the webpage content and BeautifulSoup to extract the required information from the l
# Since the website has multiple pages, I used a pagination loop (page 1 to 50) to make sure the entire book ca
# I also added a try-except block so that even if one page fails to load, the program continues scraping the re

In [1]: # Phase 1: Web Scraping - Books to Scrape
# =====
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time

# 1. Setup Lists to Store Data
titles = []
prices = []
ratings = []
availability = []
urls = []

# 2. Loop through all 50 Pages of the website
# The website has 50 pages. We loop from 1 to 51.
print("Starting Web Scraping... this may take 1-2 minutes.")

for page in range(1, 51):
    # Construct the URL for each page
    url = f"http://books.toscrape.com/catalogue/page-{page}.html"

    # Send a request to the website
    try:
        response = requests.get(url)
        response.raise_for_status() # Check for errors
    except Exception as e:
        print(f"Error on page {page}: {e}")
        continue

    # Parse the HTML content
    soup = BeautifulSoup(response.text, 'html.parser')

    # Find all book containers (articles with class 'product_pod')
    books = soup.find_all('article', class_='product_pod')

    # Loop through each book on the page and extract details
    for book in books:
        # A. Extract Title
        # Title is inside an <h3> tag, inside an <a> tag
        title = book.h3.a['title']
        titles.append(title)

        # B. Extract Price
        # Price is inside a <p> tag with class 'price_color'
        price_text = book.find('p', class_='price_color').text
        # Clean it immediately (remove '£' symbol)
        prices.append(price_text.replace('£', ''))

        # C. Extract Star Rating
        # Rating is in a <p> tag class, e.g., "star-rating Three"
        # We get the second class name (e.g., 'Three', 'Four')
        rating_class = book.find('p', class_='star-rating')['class'][1]
        ratings.append(rating_class)

```

```

# D. Extract Availability
# It's inside a <p> with class 'instock availability'
stock_text = book.find('p', class_='instock availability').text.strip()
availability.append(stock_text)

# E. URL (Useful for checking later)
book_url = "http://books.toscrape.com/catalogue/" + book.h3.a['href']
urls.append(book_url)

# Optional: Print progress every 10 pages
if page % 10 == 0:
    print(f"Scraped {page} pages...")

# 3. Create a DataFrame
df = pd.DataFrame({
    'Title': titles,
    'Price': prices,
    'Rating': ratings,
    'Availability': availability,
    'URL': urls
})

# 4. Save Raw Data to CSV
# We save this as 'raw' because we haven't cleaned it yet (Phase 2)
df.to_csv('books_scraped_data.csv', index=False)

print("\nSUCCESS! Scraping Complete.")
print(f"Total Books Scraped: {len(df)}")
print("Data saved to 'books_scraped_data.csv'")

# 5. Show the first few rows to verify
display(df.head())

```

Starting Web Scraping... this may take 1-2 minutes.

Scraped 10 pages...

Scraped 20 pages...

Scraped 30 pages...

Scraped 40 pages...

Scraped 50 pages...

SUCCESS! Scraping Complete.

Total Books Scraped: 1000

Data saved to 'books_scraped_data.csv'

	Title	Price	Rating	Availability	URL
0	A Light in the Attic	Â51.77	Three	In stock	http://books.toscrape.com/catalogue/a-light-in...
1	Tipping the Velvet	Â53.74	One	In stock	http://books.toscrape.com/catalogue/tipping-th...
2	Soumission	Â50.10	One	In stock	http://books.toscrape.com/catalogue/soumission...
3	Sharp Objects	Â47.82	Four	In stock	http://books.toscrape.com/catalogue/sharp-obje...
4	Sapiens: A Brief History of Humankind	Â54.23	Five	In stock	http://books.toscrape.com/catalogue/sapiens-a-...

In []: # Phase 2: Data Cleaning & Preprocessing

```

# The scraped data was not directly usable because most values were stored as text.
# So in this phase, I cleaned the dataset and converted important columns into proper numeric format for analys.

# Key Transformation Logic:

# Price Cleaning: Prices were in the format £51.77, so I removed the £ symbol and converted the values into flo.
# Stock Extraction (Regex): Availability was given like "In stock (22 available)", so I used re.findall() to ex
# Rating Mapping: Ratings were in text form like "One", "Two", "Five", so I mapped them into numeric values (1-

```

In [1]: # Phase 2: Data Cleaning

```

# =====

import pandas as pd
import re

# -----
# Step 1: Load the raw scraped dataset
# -----
df = pd.read_csv("books_scraped_data.csv")
print("Raw dataset loaded successfully.")
print("Dataset shape:", df.shape)

# -----
# Step 2: Clean the Price column
# -----
# Price values may contain currency symbols like £ or extra characters

```

```

# So we remove everything except digits and decimal point
df["Price"] = df["Price"].astype(str).str.replace(r"[^\d.]", "", regex=True)

# Convert cleaned price values into numeric format
df["Price"] = pd.to_numeric(df["Price"], errors="coerce")
print("Price column cleaned and converted to numeric format.")

# -----
# Step 3: Convert Rating into numeric values
# -----
# Ratings are stored as text like "One", "Two", etc.
rating_map = {
    "One": 1,
    "Two": 2,
    "Three": 3,
    "Four": 4,
    "Five": 5
}

df["Rating"] = df["Rating"].map(rating_map)
print("Rating column converted to numeric scale (1 to 5).")

# -----
# Step 4: Extract stock count from Availability column
# -----
# Availability is usually stored like: "In stock (19 available)"
# We extract the number part using regex
def extract_stock_count(value):
    value = str(value)
    stock_numbers = re.findall(r"\d+", value)

    if len(stock_numbers) > 0:
        return int(stock_numbers[0])
    return 0

df["Stock"] = df["Availability"].apply(extract_stock_count)
print("Stock count extracted successfully.")

# -----
# Step 5: Remove duplicate records
# -----
before_rows = df.shape[0]
df.drop_duplicates(inplace=True)
after_rows = df.shape[0]

print(f"Duplicates removed: {before_rows - after_rows}")

# -----
# Step 6: Create cleaned dataset and save it
# -----
# Drop the original availability column since we already extracted stock count
df_clean = df.drop(columns=["Availability"])

# Keep only the useful columns in proper order
df_clean = df_clean[["Title", "Price", "Rating", "Stock", "URL"]]

# Save the cleaned dataset
df_clean.to_csv("books_cleaned_data.csv", index=False)

print("\nData cleaning completed successfully.")
print("Cleaned file saved as: books_cleaned_data.csv")

# -----
# Step 7: Quick verification checks
# -----
print("\nFirst 5 rows of cleaned dataset:")
display(df_clean.head())

print("\nPrice column statistics:")
print(df_clean["Price"].describe())

```

Raw dataset loaded successfully.

Dataset shape: (1000, 5)

Price column cleaned and converted to numeric format.

Rating column converted to numeric scale (1 to 5).

Stock count extracted successfully.

Duplicates removed: 0

Data cleaning completed successfully.

Cleaned file saved as: books_cleaned_data.csv

First 5 rows of cleaned dataset:

	Title	Price	Rating	Stock	URL
0	A Light in the Attic	51.77	3	0	http://books.toscrape.com/catalogue/a-light-in...
1	Tipping the Velvet	53.74	1	0	http://books.toscrape.com/catalogue/tipping-th...
2	Soumission	50.10	1	0	http://books.toscrape.com/catalogue/soumission...
3	Sharp Objects	47.82	4	0	http://books.toscrape.com/catalogue/sharp-obje...
4	Sapiens: A Brief History of Humankind	54.23	5	0	http://books.toscrape.com/catalogue/sapiens-a-...

Price column statistics:

```
count    1000.000000
mean      35.07035
std       14.44669
min       10.000000
25%       22.10750
50%       35.98000
75%       47.45750
max       59.99000
```

Name: Price, dtype: float64

```
In [2]: # Phase 3: Exploratory Data Analysis (EDA)
# =====

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

# -----
# Step 1: Suppress warnings (clean output)
# -----
warnings.filterwarnings("ignore")

# -----
# Step 2: Load cleaned dataset
# -----
df = pd.read_csv("books_cleaned_data.csv")
sns.set_style("whitegrid")

print("Cleaned dataset loaded successfully.")
print("Dataset shape:", df.shape)

# -----
# Chart 1: Price Distribution
# -----
plt.figure(figsize=(10, 6))

# Histogram to understand how prices are distributed
sns.histplot(df["Price"], bins=30, kde=True)

plt.title("Distribution of Book Prices", fontsize=16)
plt.xlabel("Price (£)")
plt.ylabel("Number of Books")

# Save plot for report
plt.savefig("eda_price_dist.png")
plt.show()

# -----
# Chart 2: Count of Books by Rating
# -----
plt.figure(figsize=(8, 5))

# Countplot to see how many books belong to each star rating category
sns.countplot(
    x="Rating",
    data=df,
    hue="Rating",
    legend=False,
    palette="viridis"
)

plt.title("Number of Books per Star Rating", fontsize=16)
plt.xlabel("Star Rating")
plt.ylabel("Count")

plt.savefig("eda_rating_count.png")
plt.show()

# -----
# Chart 3: Price vs Rating
# -----
```

```
plt.figure(figsize=(10, 6))

# Boxplot to check if rating has any impact on book pricing
sns.boxplot(
    x="Rating",
    y="Price",
    data=df,
    hue="Rating",
    legend=False,
    palette="coolwarm"
)

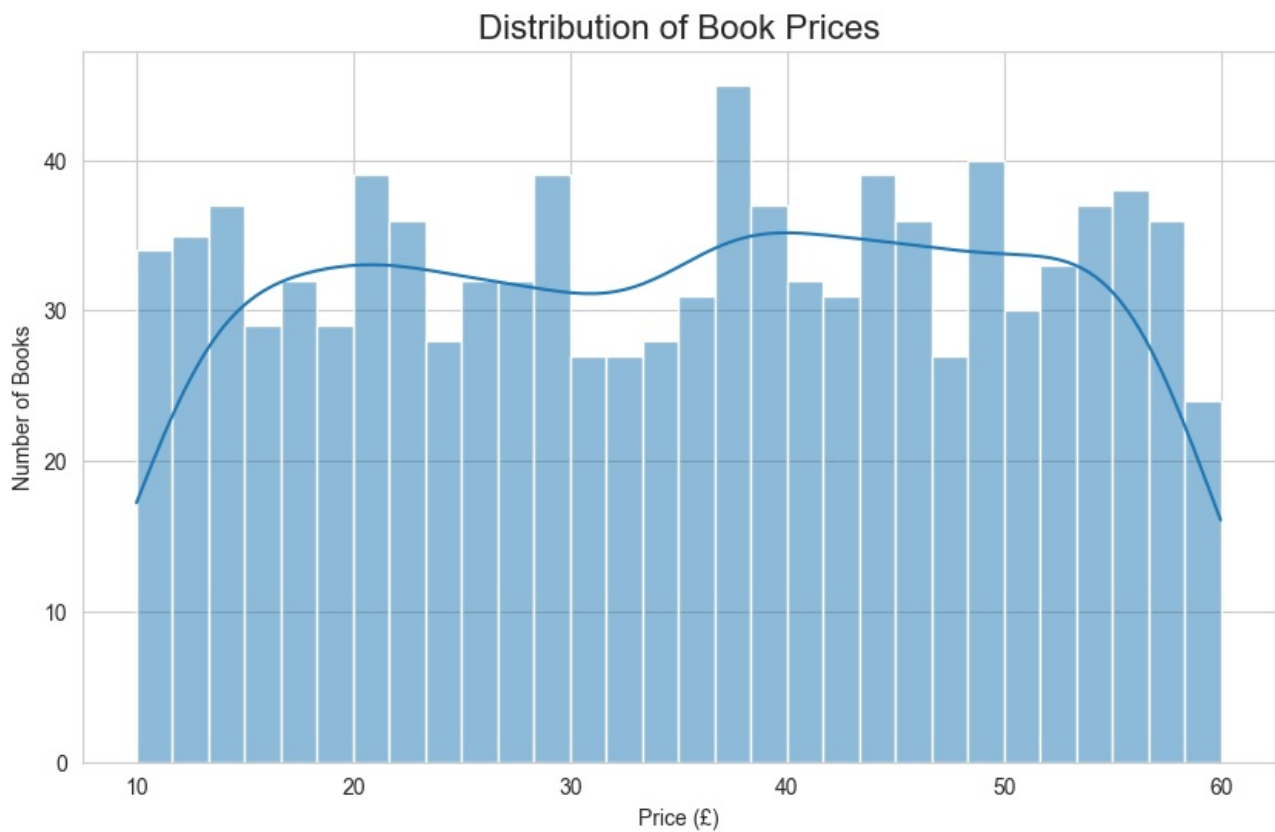
plt.title("Price Distribution by Star Rating", fontsize=16)
plt.xlabel("Star Rating")
plt.ylabel("Price (£)")

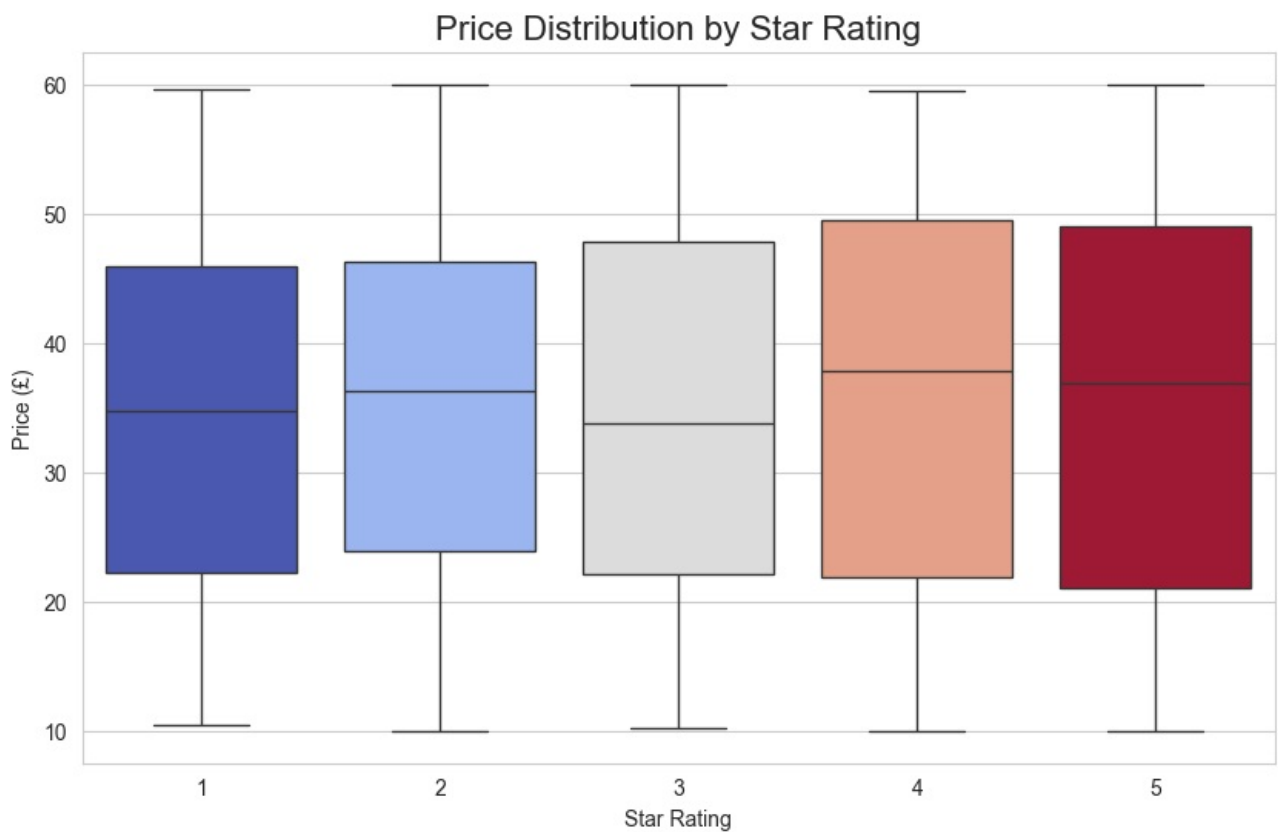
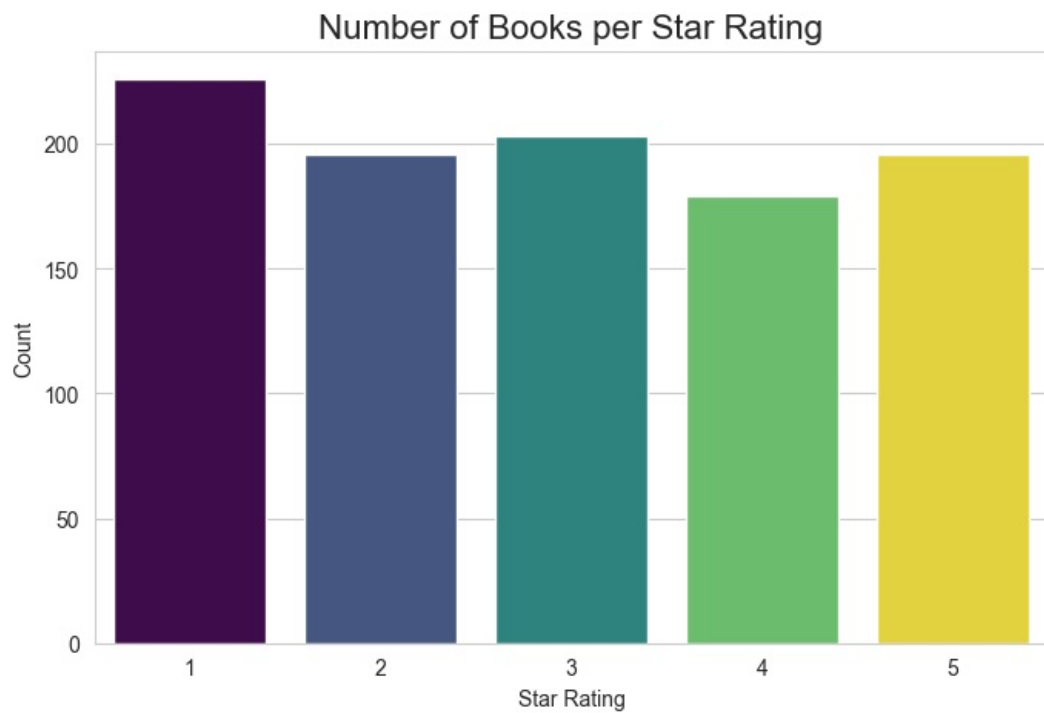
plt.savefig("eda_price_vs_rating.png")
plt.show()

# -----
# Quick Insights
# -----
rating_price_corr = df["Rating"].corr(df["Price"])

print("\n--- Quick EDA Insights ---")
print(f"Total Rows and Columns: {df.shape}")
print(f"Correlation between Rating and Price: {rating_price_corr:.4f}")
```

Cleaned dataset loaded successfully.
Dataset shape: (1000, 5)





--- Quick EDA Insights ---

Total Rows and Columns: (1000, 5)

Correlation between Rating and Price: 0.0282

```
In [ ]: # Interpretation: Price Distribution

# Observation: From the histogram, I noticed that book prices are mostly spread between £10 and £60, and the di
# Insight: This means the competitor is not focusing only on low-priced books or only premium books. Instead, ti

# Interpretation: Book Ratings

# Observation: From the rating bar chart, I can see that books are almost evenly spread across all ratings from
# Insight: In real e-commerce websites, ratings usually lean more towards 4-5 stars, but here the distribution .

# Interpretation: Price vs. Rating

# Observation: From the boxplot, I noticed that the average/median price is almost the same (around £35) for ev
# Insight: This means higher-rated books are not necessarily priced higher. So, rating does not have much effec
```

```
In [ ]: # Phase 4: Machine Learning (Price Prediction)

# In this phase, I trained a Random Forest Regressor to predict the book price using features like rating and s

# Why I used Random Forest:

# Book price may not follow a straight-line pattern, so Random Forest works better than simple Linear Regression
# Since it is an ensemble model (uses multiple decision trees), it usually gives more stable and accurate predi
# It also reduces overfitting compared to a single decision tree, which makes it a good choice for this dataset
```

```
In [3]: # Phase 4: Machine Learning Model Building
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import joblib

# -----
# Step 1: Load the cleaned dataset
# -----
df = pd.read_csv("books_cleaned_data.csv")
print("Dataset loaded for ML training.")
print("Dataset shape:", df.shape)

# -----
# Step 2: Define features and target variable
# -----
# Goal: Predict book price using Rating and Stock availability
X = df[["Rating", "Stock"]]
y = df["Price"]

print("\nSelected Features:", X.columns.tolist())
print("Target Column: Price")

# -----
# Step 3: Train-Test Split
# -----
# Using 80% data for training and 20% for testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("\nTraining set size:", X_train.shape)
print("Testing set size :", X_test.shape)

# -----
# Step 4: Train the model
# -----
# Random Forest works well for non-linear relationships
rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
```

```

rf_model.fit(X_train, y_train)
print("\nRandom Forest model training completed.")

# -----
# Step 5: Make predictions on test data
# -----
y_pred = rf_model.predict(X_test)

# -----
# Step 6: Evaluate model performance
# -----
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\n--- Model Performance Report ---")
print(f"Mean Absolute Error (MAE)      : £{mae:.2f}")
print(f"Root Mean Squared Error (RMSE)   : £{rmse:.2f}")
print(f"R-Squared Score (R²)               : {r2:.4f}")

# -----
# Step 7: Visualize Actual vs Predicted values
# -----
plt.figure(figsize=(10, 6))

# Scatter plot to compare predicted and actual prices
plt.scatter(y_test, y_pred, alpha=0.6)

# Reference line for perfect prediction
plt.plot(
    [y.min(), y.max()],
    [y.min(), y.max()],
    "r--",
    lw=2
)

plt.title("Actual vs Predicted Book Prices")
plt.xlabel("Actual Price (£)")
plt.ylabel("Predicted Price (£)")

plt.savefig("ml_prediction_chart.png")
plt.show()

# -----
# Step 8: Save the trained model (for future use)
# -----
joblib.dump(rf_model, "book_price_model.pkl")
print("\nModel saved successfully as: book_price_model.pkl")

```

Dataset loaded for ML training.
Dataset shape: (1000, 5)

Selected Features: ['Rating', 'Stock']
Target Column: Price

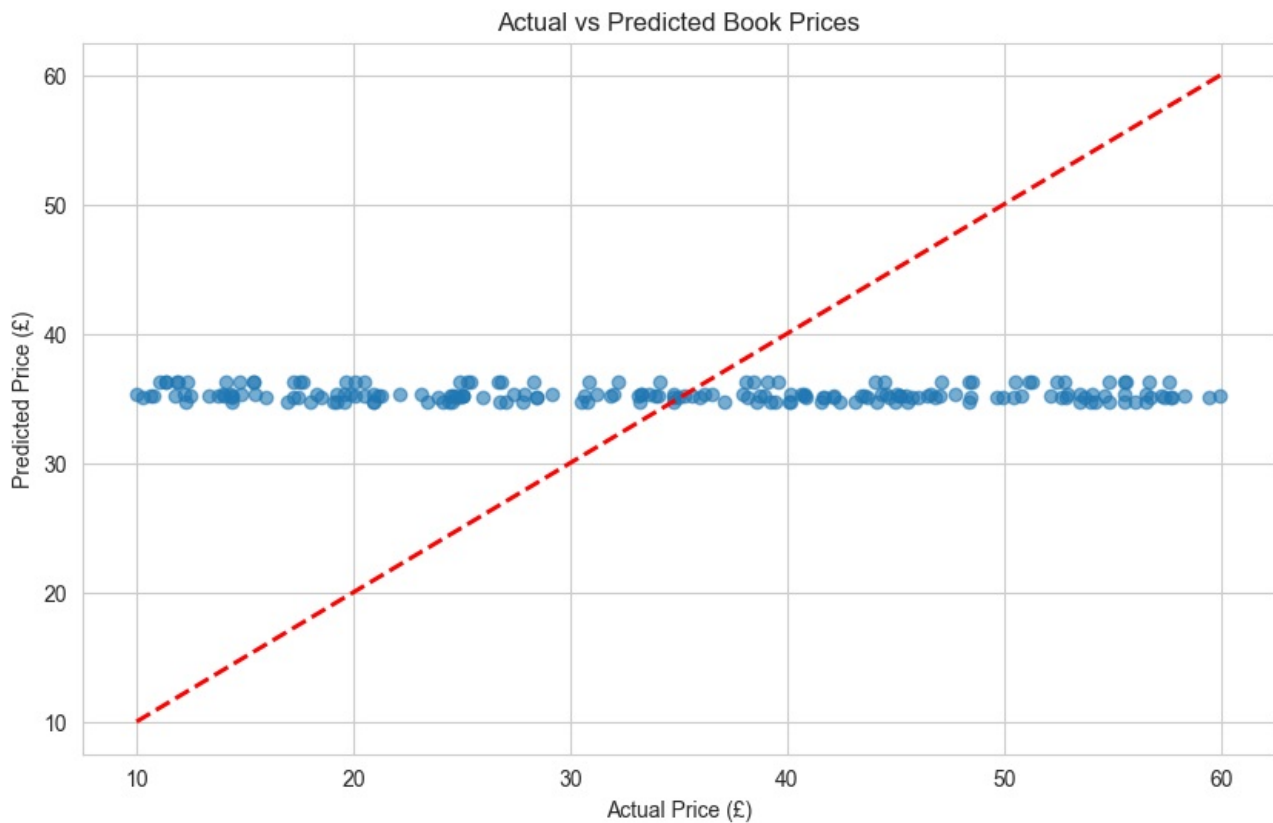
Training set size: (800, 2)
Testing set size : (200, 2)

Random Forest model training completed.

```

--- Model Performance Report ---
Mean Absolute Error (MAE)      : £12.70
Root Mean Squared Error (RMSE) : £14.59
R-Squared Score (R²)          : -0.0131

```

Model saved successfully as: book_price_model.pkl

```
In [4]: %%writefile app.py
import streamlit as st
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import joblib

# -----
# Page Configuration
# -----
st.set_page_config(page_title="Book Price Analytics", layout="wide")

sns.set_style("whitegrid")

# -----
# Load Data and Model
# -----
@st.cache_data
def load_data():
    return pd.read_csv("books_cleaned_data.csv")

@st.cache_resource
def load_model():
    return joblib.load("book_price_model.pkl")

df = load_data()
model = load_model()
```

```

# -----
# Title and Introduction
# -----
st.title("Book Price Analytics Dashboard")

st.write(
    """
    This dashboard provides a complete view of competitor book pricing behavior.
    It also includes a Machine Learning model that predicts the estimated book price based on:
    - Star Rating
    - Stock Availability
    """
)

st.markdown("---")

# -----
# Sidebar Controls
# -----
st.sidebar.header("Dashboard Controls")

# ML Prediction Tool
st.sidebar.subheader("AI Price Predictor")
st.sidebar.write("Enter book rating and stock to predict expected price.")

rating_input = st.sidebar.selectbox("Book Rating", [1, 2, 3, 4, 5], index=2)
stock_input = st.sidebar.number_input("Stock Available", min_value=0, max_value=500, value=10)

if st.sidebar.button("Predict Price"):
    input_data = pd.DataFrame([rating_input, stock_input], columns=["Rating", "Stock"])
    predicted_price = model.predict(input_data)[0]
    st.sidebar.success(f"Estimated Price: £{predicted_price:.2f}")

st.sidebar.markdown("---")

# Filters
st.sidebar.subheader("Data Filters")

selected_ratings = st.sidebar.multiselect(
    "Select Ratings",
    options=[1, 2, 3, 4, 5],
    default=[1, 2, 3, 4, 5]
)

st.sidebar.write("Price Filter (£)")
min_limit = int(df["Price"].min())
max_limit = int(df["Price"].max())

col_min, col_max = st.sidebar.columns(2)

with col_min:
    min_price = st.number_input(
        "Min Price",
        min_value=min_limit,
        max_value=max_limit,
        value=min_limit
    )

with col_max:
    max_price = st.number_input(
        "Max Price",
        min_value=min_limit,
        max_value=max_limit,
        value=max_limit
    )

# -----
# Apply Filters
# -----
filtered_df = df[
    (df["Rating"].isin(selected_ratings)) &
    (df["Price"] >= min_price) &
    (df["Price"] <= max_price)
]

# If no data after filtering
if filtered_df.empty:
    st.warning("No data available for the selected filters. Please adjust the filters.")
    st.stop()

```

```

# -----
# KPI Metrics Section
# -----
st.subheader("Key Performance Indicators")

col1, col2, col3, col4 = st.columns(4)

col1.metric("Total Books", len(filtered_df))
col2.metric("Average Price", f"£{filtered_df['Price'].mean():.2f}")
col3.metric("Total Stock", int(filtered_df["Stock"].sum()))
col4.metric("Average Rating", f"{filtered_df['Rating'].mean():.1f} / 5")

st.markdown("---")

# -----
# Market Insights Section
# -----
st.subheader("Quick Market Insights")

highest_price = filtered_df["Price"].max()
lowest_price = filtered_df["Price"].min()
most_common_rating = filtered_df["Rating"].mode()[0]
avg_stock = filtered_df["Stock"].mean()

st.write(f"- Highest price in selection: **£{highest_price:.2f}**")
st.write(f"- Lowest price in selection: **£{lowest_price:.2f}**")
st.write(f"- Most common rating: **{most_common_rating} stars**")
st.write(f"- Average stock availability: **{avg_stock:.1f} units**")

st.markdown("---")

# -----
# Charts Section
# -----
st.subheader("Market Analysis Charts")

tab1, tab2, tab3, tab4 = st.tabs(
    ["Price Distribution", "Rating Count", "Price vs Rating", "Stock vs Price"]
)

with tab1:
    st.write("### Distribution of Book Prices")
    fig, ax = plt.subplots(figsize=(10, 4))
    sns.histplot(filtered_df["Price"], bins=20, kde=True, ax=ax)
    ax.set_xlabel("Price (£)")
    ax.set_ylabel("Number of Books")
    st.pyplot(fig)

with tab2:
    st.write("### Count of Books by Rating")
    fig, ax = plt.subplots(figsize=(10, 4))
    sns.countplot(x="Rating", data=filtered_df, ax=ax, palette="viridis")
    ax.set_xlabel("Star Rating")
    ax.set_ylabel("Count")
    st.pyplot(fig)

with tab3:
    st.write("### Price Spread by Star Rating")
    fig, ax = plt.subplots(figsize=(10, 4))
    sns.boxplot(x="Rating", y="Price", data=filtered_df, ax=ax, palette="coolwarm")
    ax.set_xlabel("Star Rating")
    ax.set_ylabel("Price (£)")
    st.pyplot(fig)

with tab4:
    st.write("### Stock vs Price Relationship")
    fig, ax = plt.subplots(figsize=(10, 4))
    sns.scatterplot(
        x="Stock",
        y="Price",
        hue="Rating",
        data=filtered_df,
        ax=ax,
        palette="deep",
        alpha=0.7
    )
    ax.set_xlabel("Stock Available")
    ax.set_ylabel("Price (£)")
    ax.set_title("Price vs Stock (Colored by Rating)")

```

```

        st.pyplot(fig)

st.markdown("---")

# -----
# Feature Importance Section (Professional Touch)
# -----
st.subheader("Model Explanation (Feature Importance)")

if hasattr(model, "feature_importances_"):
    importance_df = pd.DataFrame({
        "Feature": ["Rating", "Stock"],
        "Importance": model.feature_importances_
    }).sort_values(by="Importance", ascending=False)

    fig, ax = plt.subplots(figsize=(7, 4))
    sns.barplot(x="Importance", y="Feature", data=importance_df, ax=ax, palette="Blues_r")
    ax.set_title("Random Forest Feature Importance")
    st.pyplot(fig)

    st.write("This chart shows which feature contributes more to predicting the book price.")
else:
    st.info("Feature importance is only available for tree-based models like Random Forest.")

st.markdown("---")

# -----
# Top 10 Expensive Books Section
# -----
st.subheader("Top 10 Most Expensive Books (Filtered Data)")

top_10_books = filtered_df.sort_values(by="Price", ascending=False).head(10)

fig, ax = plt.subplots(figsize=(12, 5))
sns.barplot(x="Price", y="Title", data=top_10_books, ax=ax, palette="magma")
ax.set_title("Highest Priced Books in Selected Data")
ax.set_xlabel("Price (£)")
ax.set_ylabel("Book Title")
st.pyplot(fig)

st.markdown("---")

# -----
# Dataset Preview + Download
# -----
st.subheader("Filtered Dataset Preview")

left_col, right_col = st.columns([3, 1])

with left_col:
    st.dataframe(filtered_df.head(100), use_container_width=True)

with right_col:
    st.write("Download filtered dataset:")
    csv_data = filtered_df.to_csv(index=False).encode("utf-8")

    st.download_button(
        label="Download CSV",
        data=csv_data,
        file_name="filtered_books_data.csv",
        mime="text/csv"
    )

# -----
# Footer
# -----
st.markdown("---")
st.caption("Developed By : Preetham Sharvin Danthi")

```

Overwriting app.py

In []: !streamlit run app.py

In []: